[Huyen Nguyen] Threads & Synchronization

1. Thread

1.1. Thread Concepts

A **thread** is the smallest unit of execution within a process. Each thread has its own:

- **Program Counter (PC)**
- **Register set**
- **Stack**

But threads within the same process **share**:

- Address space (heap)
- Code segment
- Global data
- File descriptors and other system resources

Process vs Thread comparison:

Criteria	Process	Thread
Memory	Independent	Shared with other threads
Creation cost	High	Low
Communication	IPC (complex)	Shared variables (simple, but error-prone)
Context switch	Slow	Fast

Why use threads?

- Take advantage of multi-core CPUs (parallelism)
- Make programs more responsive (e.g., a separate UI thread, a separate background-processing thread)
- More resource-efficient than spawning multiple processes

1.2. Thread Identification

Each thread has a **Thread ID (TID)** so the OS and the program can distinguish between threads.

Example using POSIX threads (pthread) in C:

```
#include <pthread.h>
#include <stdio.h>
void *task(void *arg) {
    pthread_t tid = pthread_self();
    printf("Running thread ID: %lu\n", (unsigned long)tid);
    return NULL;
}
int main() {
    pthread_t t1;
    pthread_create(&t1, NULL, task, NULL);
    printf("Main created thread ID: %lu\n", (unsigned long)t1);
    pthread_join(t1, NULL);
    return 0;
}
```

Output:

```
Main created thread ID: 129812588918464
Running thread ID: 129812588918464
```

- `pthread_self()`: returns the ID of the currently running thread
- `pthread_equal(t1, t2)`: checks whether two thread IDs refer to the same thread

1.3. Thread Creation

Main function: `pthread_create()`

```
int pthread_create(
    pthread_t *thread,              /* pointer to store the new thread's ID */
    const pthread_attr_t *attr,     /* attributes (NULL = default) */
    void *(*start_routine)(void*),  /* function the thread will run */
    void *arg                       /* argument passed to the function */
);
```

Example — creating multiple threads:

```
#include <pthread.h>
#include <stdio.h>
void *worker(void *arg) {
    int id = *(int *)arg;
    printf("Thread %d is running\n", id);
    return NULL;
}
int main() {
    pthread_t threads[3];
    int ids[3] = {1, 2, 3};
    for (int i = 0; i < 3; i++) {
        pthread_create(&threads[i], NULL, worker, &ids[i]);
    }
    for (int i = 0; i < 3; i++) {
        pthread_join(threads[i], NULL);
    }
    printf("All threads have finished!\n");
    return 0;
}
```

Output:

```
Thread 1 is running
Thread 3 is running
Thread 2 is running
All threads have finished!
```

Note: In the example above, passing `&ids[i]` (a separate variable per thread) is safer than passing `&i` directly, because the value of `i` changes before a thread gets a chance to read it — leading to a race condition.

- If `&i` is passed instead:

```
for (int i = 0; i < 3; i++) {
    pthread_create(&threads[i], NULL, worker, &i);
}
```

Output:

```
Thread 2 is running
Thread 2 is running /* Slot 2 got overwritten */
```

```
Thread 3 is running
All threads have finished!
```

1.4. Thread Termination

A thread terminates when:

- Its function finishes (returns)
- It exits actively and can return a value
- It is cancelled by another thread via `pthread_cancel(pthread_t thread)`
- The entire process terminates

Example:

```
#include <pthread.h>
#include <stdio.h>
#include <stdlib.h>
void *task(void *arg) {
    int *result = malloc(sizeof(int));
    *result = 999;
    printf("Child thread: %d\n", *result);
    pthread_exit((void *)result);
}
int main() {
    pthread_t t;
    void *result;
    pthread_create(&t, NULL, task, NULL);
    pthread_join(t, &result);
    printf("Result: %d\n", *(int *)result);
    free(result);
    return 0;
}
```

Output:

```
Child thread: 999
Result: 999
```

- **`pthread_join()`**: blocks until the specified thread finishes — used for synchronization and to avoid "zombie threads"
- **`pthread_detach()`**: marks a thread to release its own resources automatically when it finishes, without needing to be joined

2. Why Is Concurrency Hard? Race Conditions & Mutual Exclusion

2.1. Race Condition

A race condition occurs when **two or more threads access a shared variable/resource concurrently**, and **the final result depends on the order of execution** (which is unpredictable).

Example: two threads incrementing a shared counter `counter++`

Lab 1.

```
#include <pthread.h>
#include <stdio.h>
int counter = 0;
void *increment(void *arg) {
    for (int i = 0; i < 1000000; i++) {
        counter++;
    }
    return NULL;
}
int main() {
    pthread_t t1, t2;
    pthread_create(&t1, NULL, increment, NULL);
    pthread_create(&t2, NULL, increment, NULL);
    pthread_join(t1, NULL);
    pthread_join(t2, NULL);
    printf("EXPECTED result (1,000,000 * 2) = 2000000\n");
    printf("ACTUAL result   = %d\n", counter);
    return 0;
}
```

The `counter++` instruction actually consists of 3 steps at the CPU level:

- LOAD counter -> register
- ADD register + 1
- STORE register -> counter

Output:

```
EXPECTED result (1,000,000 * 2) = 2000000
ACTUAL (incorrect) result       = 1612870
```

If two threads interleave these three steps, some increments get "lost" → the final value of `counter` ends up lower than expected (unpredictable, and different on each run).

→ This is why concurrency is hard: **the bug is non-deterministic** and hard to debug — the program might run correctly 99 times and then fail on the 100th.

2.2. Critical Section

A **critical section** is a piece of code that accesses a shared resource (a variable, a file, a device, etc.). At any given moment, it must be guaranteed that **only one thread** is executing inside the critical section.

3 requirements for a correct solution to the Critical Section Problem:

1. **Mutual Exclusion**: only one thread may be in the critical section at a time → prevents race conditions
2. **Progress**: if no thread is in the critical section, some thread must eventually be allowed to enter (no infinite waiting) → prevents deadlock
3. **Bounded Waiting**: each thread only waits a limited amount of time → prevents starvation

2.3. Mutual Exclusion — Solving It with a Mutex

A **mutex (Mutual Exclusion Lock)** is the most common tool for protecting a critical section.

Lab 2.

```
#include <pthread.h>
#include <stdio.h>
int counter = 0;
pthread_mutex_t lock = PTHREAD_MUTEX_INITIALIZER;
void *increment(void *arg) {
    for (int i = 0; i < 100000; i++) {
        pthread_mutex_lock(&lock);
        counter++;
        pthread_mutex_unlock(&lock);
    }
    return NULL;
}
int main() {
```

```
    pthread_t t1, t2;
    pthread_create(&t1, NULL, increment, NULL);
    pthread_create(&t2, NULL, increment, NULL);
    pthread_join(t1, NULL);
    pthread_join(t2, NULL);
    pthread_mutex_destroy(&lock);
    printf("EXPECTED result = 200000\n");
    printf("ACTUAL result   = %d \n", counter);
    return 0;
}
```

Output:

```
EXPECTED result = 200000
ACTUAL result   = 200000
```

- `pthread_mutex_lock()`: if the mutex is already locked by another thread, the current thread **blocks** (waits) until it is released
- `pthread_mutex_unlock()`: releases the lock, allowing another thread to enter

2.4. Other Synchronization Tools

Tool	Purpose
Mutex	Ensures mutual exclusion for a single resource
Semaphore	Controls the number of threads allowed to access a resource (a counted lock)
Condition Variable	Lets a thread wait until some condition becomes true before continuing
Spinlock	Similar to a mutex, but waits by "busy-looping" instead of sleeping — used when wait times are very short

2.5. Problems That Arise From Misusing Locks

- **Deadlock**: two or more threads each wait for a lock held by the other → permanently stuck
- **Starvation**: a thread is repeatedly "cut in line" and never gets to enter the critical section
- **Priority Inversion**: a low-priority thread holds a lock that a high-priority thread needs, delaying the high-priority thread

3. Case Study: The Mars Pathfinder Priority Inversion Bug (1997)

3.1. Background

In July 1997, the **Mars Pathfinder** lander successfully touched down on Mars. A few days after it began transmitting data back to Earth, the system started experiencing unpredictable **total system resets**, causing loss of data and valuable observation time.

The real-time operating system (RTOS) used was **VxWorks**, running a **priority-based preemptive scheduler** — a higher-priority task always preempts a lower-priority one.

3.2. The Three Task Involved

Task	Priority	Role
Bus Management Task (BMT)	High	Manages access to the "information bus" — a shared data bus
Communications Task (COMM)	Medium	Handles communication with Earth
Meteorological Task	Low	Collects weather data, writes it to the bus via a **mutex**

BMT and the Meteorological Task shared a **mutex** to protect the information bus.

3.3. Sequence of Events That Caused the Bug

The **low**-priority task (Meteorological) acquires the mutex to write data to the bus.

Meanwhile, the **high**-priority task (BMT) needs access to the same bus → it gets **blocked**, waiting for the mutex to be released.

The **medium**-priority task (COMM) — unrelated to the mutex — starts running and **preempts** the low-priority task (since COMM has higher priority than Meteorological).

Because the low-priority task is "frozen" by the medium-priority task, it cannot resume and **release the mutex**.

Result: the **highest**-priority task (BMT) ends up indirectly blocked by a **medium**-priority task, even though BMT should, in principle, always win over COMM. This is exactly **priority inversion**.

BMT waits too long → the watchdog timer detects that BMT "failed to complete its work on schedule" → triggers a **full system reset**.

-> What made this dangerous is that the bug **did not occur consistently** — only when the timing of these three tasks overlapped in just the wrong sequence — making it very hard to reproduce and debug.

3.4. Diagnosis & Fix

JPL engineers (with consultation from Professor David Wilner) did the following:

- Enabled **verbose logging** on a ground-based replica of the system to reproduce the bug.
- Identified the root cause as priority inversion on the mutex protecting the information bus.
- **Enabled the "priority inheritance" flag**, which already existed in VxWorks but had been left off (due to concerns about overhead) → fixed the bug **remotely, over the radio link**, with no new hardware and no need to bring the spacecraft back.

Priority Inheritance Protocol : when a high-priority task is blocked on a mutex held by a low-priority task, that low-priority task is **temporarily boosted to the priority level of the task waiting on it**. This way, an intermediate task (COMM) no longer has a chance to cut in, and the low-priority task quickly finishes and releases the mutex.

(Note: this is a simplified illustration of the mechanism, not a verbatim excerpt of the actual VxWorks flight source code.)

```
give_mutex_priority_inheritance = 1; /* Flip the flag from FALSE (0) to TRUE (1) */
mutex_bus_1553->options |= SEM_INVERSION_SAFE; /* Enable the inversion-safe option on the
mutex */
```

3.5. Lessons on Shared-Resource Locking

Priority inversion is an inherent risk, not a rare edge case, whenever mutexes/locks are used in a system with multiple priority levels — any real-time or multi-threaded system with shared resources is exposed to it.

Looking only at the two tasks directly involved with the lock is not enough. The bug only surfaces when a **third, unrelated task** cuts in at just the right moment — so you must analyze the full set of concurrently running tasks, not just the lock holder/waiter pair.

Priority Inheritance / Priority Ceiling Protocol should be enabled by default in real-time systems with strict timing constraints, unless there's a clear reason to disable it.

Testing under realistic, sustained load matters more than testing modules in isolation — this bug only appeared under specific load and timing conditions that are hard to catch in unit tests.

Design for remote diagnosability: the fact that JPL could enable logging and fix the bug over the radio, with no hardware changes, is why Pathfinder's mission was saved — a broader lesson in resilience for embedded/IoT system design.

3.6. Practical Takeaway

If applied to an embedded/IoT system:

- Any mutex/semaphore shared between tasks of different priorities should be considered for `configUSE_MUTEXES` + `configUSE_PRIORITY_INHERITANCE` in FreeRTOS.
- Without priority inheritance, **minimize the time a lock is held** (keep critical sections as short as possible) and avoid letting a low-priority task hold a lock while a medium-priority task that can run for a long time is active.

4. Thread Synchronization Basics: Mutex Locks and Critical Sections

4.1. What Is a Critical Section?

A **Critical Section** is a piece of code that **accesses a shared resource**, and must guarantee that **only one thread executes it at a time**.

Requirements for a good critical-section synchronization solution:

Requirement	Meaning
Mutual Exclusion	Only one thread may be inside the critical section at a time
Progress	If no thread is currently inside the CS, a thread wanting to enter must be allowed to (no unreasonable blocking)
Bounded Waiting	A waiting thread must not be starved indefinitely

4.2. Mutex Lock — the Most Basic Tool

A **Mutex** (Mutual Exclusion Lock) is a synchronization object with two states — **locked** / **unlocked** — used to protect a critical section.

Basic mechanism:

```
mutex_lock(&m);     /* If the mutex is unlocked → lock it, the thread continues*/
                     /* If it's locked (held by another thread) → the current thread BLOCKS
and waits*/


mutex_unlock(&m);   /* Unlock, wake up one waiting thread (if any)*/
```

```
int counter = 0;
mutex_t m;

void task() {
    mutex_lock(&m);
    counter++;        /* Now protected — only one thread executes this at a time */
    mutex_unlock(&m);
}
```

4.3. Mutex vs. Semaphore

A commonly confused point:

	Mutex	Semaphore (counting)
Value	Binary only (0/1)	A countable integer (0..N)
Ownership	Whoever `lock`s it must be the one who `unlock`s it	No concept of "ownership" — thread A can `wait`, thread B can `signal`
Purpose	Protects **mutual exclusion** for one resource	Limits the **number** of threads concurrently accessing N identical resources
Typical use	Protecting a single shared variable	Capping connections to a resource pool at N

→ If you only need to protect one resource from concurrent access, a **mutex** is the right tool — not a semaphore.

4.4. Deadlock — the Biggest Risk of Using Mutexes

Deadlock happens when two (or more) threads **wait on each other indefinitely**.

Example — deadlock with two mutexes:

```
/* Thread A*/
mutex_lock(&m1);
mutex_lock(&m2);   /* A holds m1, waiting on m2*/
...

/* Thread B*/
mutex_lock(&m2);
mutex_lock(&m1);   /* B holds m2, waiting on m1*/
...
```

If A holds `m1` and waits on `m2`, while B holds `m2` and waits on `m1` → both are stuck forever.

Common prevention strategies:

- **Always lock mutexes in the same fixed order** everywhere in the code (e.g., always `m1` before `m2`).
- Keep critical sections **as short as possible** — minimize lock-holding time.
- Avoid calling unfamiliar functions (which might lock another mutex internally) while already holding a lock.
- Use `trylock` with a timeout instead of blocking `lock` indefinitely, where possible.

Example — same lock order, so no deadlock occurs:

```
pthread_mutex_t m1 = PTHREAD_MUTEX_INITIALIZER;
pthread_mutex_t m2 = PTHREAD_MUTEX_INITIALIZER;
void *thread_A(void *arg) {
    pthread_mutex_lock(&m1);
    printf("[Thread A] HOLDING LOCK m1\n");
    sleep(1);
    pthread_mutex_lock(&m2);
    printf("[Thread A] HOLDING LOCK m2\n");
    printf("[Thread A] Done\n");
    pthread_mutex_unlock(&m2);
    pthread_mutex_unlock(&m1);
    return NULL;
}
void *thread_B(void *arg) {
    pthread_mutex_lock(&m1);
    printf("[Thread B] HOLDING LOCK m1\n");
    sleep(1);
    pthread_mutex_lock(&m2);
    printf("[Thread B] HOLDING LOCK m2\n");
    printf("[Thread B] Done\n");
    pthread_mutex_unlock(&m1);
    pthread_mutex_unlock(&m2);
    return NULL;
}
int main() {
    pthread_t t1, t2;
    pthread_create(&t1, NULL, thread_A, NULL);
    pthread_create(&t2, NULL, thread_B, NULL);
    pthread_join(t1, NULL);
```

```
    pthread_join(t2, NULL);
    return 0;
}
```

Output:

```
[Thread A] HOLDING LOCK m1
[Thread A] HOLDING LOCK m2
[Thread A] Done
[Thread B] HOLDING LOCK m1
[Thread B] HOLDING LOCK m2
[Thread B] Done
```

5. Reader-Writer Locks

5.1. Why Do We Need an RW Lock Instead of a Regular Mutex?

With a regular `mutex`, only **one thread** can be in the critical section at a time — even when multiple threads only want to **read** data (without changing anything). This wastes performance unnecessarily, because:

- Multiple threads **reading** shared data at the same time do **not** conflict with each other (the data doesn't change)
- Only when a thread wants to **write** does it need to be guaranteed that no one else is reading or writing at the same time

The **Reader-Writer Lock (RW Lock)** was created to solve this problem, allowing:

- **Multiple readers** to read concurrently
- **Only one writer** to write at a time, and while a writer is writing, **no one else** (including readers) can access the data

5.2. How It Works

Current state	New reader request	New writer request
No one accessing	✅ Allowed in	✅ Allowed in
A reader is reading	✅ Allowed in (reads alongside)	Must wait
A writer is writing	Must wait	Must wait

→ A writer holds **exclusive** access, while readers hold **shared** access.

5.3. Example

```
int shared_data = 0;
pthread_rwlock_t rwlock;
void *reader(void *arg) {
    int id = *(int *)arg;
    pthread_rwlock_rdlock(&rwlock);
    printf("Reader %d reads data: %d\n", id, shared_data);
    sleep(1);
    pthread_rwlock_unlock(&rwlock);
    return NULL;
}
void *writer(void *arg) {
    int id = *(int *)arg;
    pthread_rwlock_wrlock(&rwlock);
    shared_data++;
    printf("Writer %d writes new data: %d\n", id, shared_data);
    sleep(1);
    pthread_rwlock_unlock(&rwlock);
    return NULL;
}
int main() {
    pthread_t r[3], w[1];
    int rid[3] = {1, 2, 3};
    int wid[1] = {1};
    pthread_rwlock_init(&rwlock, NULL);
    for (int i = 0; i < 3; i++)
        pthread_create(&r[i], NULL, reader, &rid[i]);
    pthread_create(&w[0], NULL, writer, &wid[0]);
    for (int i = 0; i < 3; i++) pthread_join(r[i], NULL);
    pthread_join(w[0], NULL);
    pthread_rwlock_destroy(&rwlock);
    return 0;
}
```

Output:

```
Reader 1 reads data: 0
Reader 2 reads data: 0
Reader 3 reads data: 0
```

```
Writer 1 writes new data: 1
```

5.4. Mutex vs RW Lock

Criteria	Mutex	RW Lock
Concurrent readers	1	Many
Concurrent writers	1	1
Read + Write at the same time	No	No
Best suited when	Reads and writes happen about equally often	**Reads are frequent, writes are rare** (e.g., caches, system configuration, lookup tables)

→ An RW Lock only really outperforms a mutex when **reads far outnumber writes**. If writes happen often, an RW Lock provides no meaningful benefit (it can even have higher management overhead than a mutex).

5.5. The Starvation Problem in RW Locks

This is an important point to keep in mind: an RW Lock can cause **starvation** for one side or the other, depending on the priority policy used:

- **Reader-priority:** if new readers keep arriving continuously, a writer may have to **wait indefinitely**, since there is never a moment with "no readers" → the writer starves
- **Writer-priority:** conversely, once a writer is waiting, any new readers that arrive afterward are blocked so the writer can go first → this prevents writer starvation, but can reduce read throughput
- **Fair RW Lock (FIFO):** requests are served in the order they arrive — commonly used in real systems to avoid both types of starvation

6. Reentrancy and Thread-Specific Data

6.1. Reentrancy

Concept

A function is called reentrant if it can be called concurrently by multiple threads (or interrupted and called again recursively) and still produce correct results, without causing data conflicts.

Conditions for a function to be reentrant:

- Does not use static variables or global variables to store shared state
- Does not return a pointer to a static memory region inside the function

- Only operates on local data (local variables on the stack) or data passed in through parameters
- Does not call other non-reentrant functions

Example of a NON-reentrant function

```
#include <stdio.h>

char buffer[100];

char *format_message(const char *name) {
    sprintf(buffer, "Hello, %s!", name);
    return buffer;
}
```

If 2 threads call `format_message()` at nearly the same time, thread A just writes "Hello, An!" into `buffer`, then thread B overwrites it with "Hello, Binh!" before A can read/print it → A ends up reading B's data by mistake.

The `strtok()` function in C uses an internal static variable to remember the current parsing position → not reentrant, not thread-safe.

Example of a reentrant function

```
#include <stdio.h>
#include <pthread.h>
int number(int *count) {
    (*count)++;
    return *count;
}
void *ham_1(void *arg) {
    int my_counter = 0;
    for (int i = 0; i < 3; i++) {
        printf("[Ham 1] Number: %d\n", number(&my_counter));
    }
    return NULL;
}
void *ham_2(void *arg) {
    int my_counter = 0;
    for (int i = 0; i < 3; i++) {
        printf("[Ham 2 ] Number: %d\n", number(&my_counter));
    }
```

```
    return NULL;
}
int main() {
    pthread_t t1, t2;
    pthread_create(&t1, NULL, ham_1, NULL);
    pthread_create(&t2, NULL, ham_2, NULL);
    pthread_join(t1, NULL);
    pthread_join(t2, NULL);
    return 0;
}
```

Output:

```
[Ham 1] Number: 1
[Ham 1] Number: 2
[Ham 2 ] Number: 1
[Ham 2 ] Number: 2
[Ham 2 ] Number: 3
[Ham 1] Number: 3
```

→ Each call uses its own memory on each thread's own stack, so there is no longer any conflict.

Note on the distinction:

- Reentrant ≠ Thread-safe, but they are closely related:
- Reentrant usually implies thread-safe (if no shared resource is used, then multiple threads calling at the same time is still safe)
- But a function can be thread-safe without being reentrant if it uses a mutex to lock around a static variable (safe with multiple threads, but not safe if called recursively/interrupted within itself — this can easily cause a deadlock)

Example (`strtok_r` is the reentrant version of `strtok`, taking an additional `saveptr` parameter to store state instead of using an internal static variable):

```
char *strtok_r(char *str, const char *delim, char **saveptr);
```

Example of a function using a Mutex but that is non-reentrant -> Deadlock

```
#include <stdio.h>
#include <signal.h>
#include <unistd.h>
#include <pthread.h>
```

```
pthread_mutex_t lock = PTHREAD_MUTEX_INITIALIZER;

void important_work(const char *caller) {
    printf("[%s] Requesting the Mutex lock...\n", caller);
    pthread_mutex_lock(&lock);
    printf("[%s] Holding the lock\n", caller);
    sleep(3);
    pthread_mutex_unlock(&lock);
    printf("[%s] Finished work and released the lock!\n", caller);
}
void my_signal_handler(int sig) {
    printf("\nSignal received\n");
    important_work("SIGNAL HANDLER");
}
int main() {
    signal(SIGALRM, my_signal_handler);
    alarm(1);
    important_work("MAIN FUNCTION");
    return 0;
}
```

→ The handler waits for main to release the lock, while main waits for the handler to finish before it can continue and release the lock → both wait for each other forever = deadlock.

Lab 3.

```
#include <stdio.h>
#include <signal.h>
#include <unistd.h>
#include <pthread.h>

pthread_mutex_t lock = PTHREAD_MUTEX_INITIALIZER;

volatile sig_atomic_t got_signal = 0;

void my_signal_handler(int sig) {
    got_signal = 1;
}
```

```
void important_work() {
    pthread_mutex_lock(&lock);
    printf("Doing important work...\n");
    sleep(3);
    pthread_mutex_unlock(&lock);
}

int main() {
    signal(SIGALRM, my_signal_handler);
    alarm(1);

    important_work();

    if (got_signal) {
        printf("A signal was detected earlier, now handling the signal's work...\n");
        important_work();
    }

    pthread_mutex_destroy(&lock);
    return 0;
}
```

Output:

```
Doing important work...
A signal was detected earlier, now handling the signal's work...
Doing important work...
```

6.2. Thread-Specific Data (TSD) — Per-Thread Data

Sometimes we need a variable that behaves "like a global" — that is, accessible from many functions without being passed as a parameter — but we want each thread to have its own copy, not shared with other threads.

Real-world example: `errno` in C — each thread needs its own `errno`; otherwise, if thread A's call fails but hasn't read `errno` yet, thread B may already have overwritten it with a different value.

Thread-Specific Data (TSD), also called Thread-Local Storage (TLS), solves this problem: the variable has only "one name" in the code, but each thread has its own independent copy of that variable.

Method 1: pthread TSD API

```c
#include <pthread.h>
#include <stdio.h>
#include <stdlib.h>
pthread_key_t key;
void destructor(void *value) {
    printf("Giai phong bo nho cho thread\n");
    free(value);
}
void *task(void *arg) {
    int *counter = malloc(sizeof(int));
    *counter = *(int *)arg;
    pthread_setspecific(key, counter);
    int *my_counter = (int *)pthread_getspecific(key);
    printf("Thread doc gia tri rieng cua no: %d\n", *my_counter);
    return NULL;
}
int main() {
    pthread_t t1, t2;
    int a = 100, b = 200;
    pthread_key_create(&key, destructor);
    pthread_create(&t1, NULL, task, &a);
    pthread_create(&t2, NULL, task, &b);
    pthread_join(t1, NULL);
    pthread_join(t2, NULL);
    pthread_key_delete(key);
    return 0;
}
```

Output

```
Thread doc gia tri rieng cua no: 200
Giai phong bo nho cho thread
Thread doc gia tri rieng cua no: 100
Giai phong bo nho cho thread
```

- `pthread_key_create(&key, destructor)`: creates a shared "key"; `destructor` will automatically be called to free memory when the thread ends
- `pthread_setspecific(key, value)`: assigns a value specific to the current thread for that `key`
- `pthread_getspecific(key)`: retrieves the value specific to the current thread for that `key`

Method 2: `__thread` / `thread_local`

Modern compilers (GCC, Clang, C11) support direct declaration:

```
#include <pthread.h>
#include <stdio.h>
__thread int counter = 0; /*each thread has its own copy of this variable*/
void *task(void *arg) {
    counter = *(int *)arg; /* assigns to this thread's own copy, does not affect other
threads*/
    printf("Thread co counter rieng = %d\n", counter);
    return NULL;
}
int main() {
    pthread_t t1, t2;
    int a = 111, b = 222;
    pthread_create(&t1, NULL, task, &a);
    pthread_create(&t2, NULL, task, &b);
    pthread_join(t1, NULL);
    pthread_join(t2, NULL);
    return 0;
}
```

→ This approach is much easier to use than the pthread TSD API (no need for a `key`, no need to call `get/set`), but it doesn't have an automatic destructor mechanism as sophisticated as method 1.

Comparing the two approaches

Criteria	pthread TSD API	`__thread` / `thread_local`
Complexity	Higher (key, get/set)	Lower, used like a normal variable
Automatic cleanup	Yes (destructor)	Not built-in
Portability	POSIX standard	Compiler-dependent (though C11 has standardized `_Thread_local`)
When to use	Need complex cleanup logic, legacy libraries	Most common cases

7. Threads and Signals, Threads and Fork, Threads and I/O

7.1. Threads and Signals

Who does a signal belong to in a multi-threaded program?

In a single-threaded program, signals are simple: the program is interrupted, the handler runs, then it returns. But in a multi-threaded program, the question arises: which thread receives the signal and runs the handler?

According to the POSIX standard:

- Some signals (like `SIGSEGV`, `SIGFPE` — errors caused by the thread itself) are delivered directly to the thread that caused the error
- Other signals (like `SIGINT` from `Ctrl+C`, `SIGTERM`) are delivered to the entire process, and the OS RANDOMLY picks one of the threads that hasn't blocked that signal to handle it — it's not always the main thread!

Controlling signals in threads with `pthread_sigmask`

Since we can't know in advance which thread will receive a signal, a common technique is: block the signal in all threads, and create one dedicated thread whose sole job is to handle signals.

```
#include <pthread.h>
#include <signal.h>
#include <stdio.h>
#include <unistd.h>
void *signal_handler_thread(void *arg) {
    sigset_t *set = (sigset_t *)arg;
    int sig;
    while (1) {
        sigwait(set, &sig);
        printf("Thread rieng nhan duoc signal: %d\n", sig);
    }
    return NULL;
}
int main() {
    sigset_t set;
    sigemptyset(&set);
    sigaddset(&set, SIGINT); // will handle SIGINT (Ctrl+C)

    /* Block this signal in ALL threads (including main), since child threads inherit this
mask*/
    pthread_sigmask(SIG_BLOCK, &set, NULL);
    pthread_t tid;
```

```
    pthread_create(&tid, NULL, signal_handler_thread, &set);
    printf("Chuong trinh dang chay, nhan Ctrl+C de test...\n");
    while (1) sleep(1);
    return 0;
}
```

Output

```
Chuong trinh dang chay, nhan Ctrl+C de test...
^CThread rieng nhan duoc signal: 2
```

- `pthread_sigmask()`: similar to `sigprocmask()` but applies per individual thread (each thread has its own independent signal mask)
- `sigwait()`: waits for a signal in a synchronous manner (instead of using an asynchronous handler) — much safer, because the code runs like a normal function and is not restricted by "async-signal-safe" rules like a traditional signal handler is

Why is this approach safer than a traditional signal handler?

This connects directly to the issue analyzed in the previous document (`SIGALRM` + mutex → deadlock): a traditional signal handler can interrupt any thread, which is dangerous if that thread happens to be holding a mutex. Using `sigwait()` in one dedicated thread provides these benefits:

- You know exactly which thread handles the signal
- The signal-handling code runs normally (free to call `printf`, `malloc`, use mutexes, etc.), without the restrictions that apply inside a real signal handler

`pthread_kill(thread, sig)`: sends a signal to one specific thread (different from `kill(pid, sig)`, which sends to the entire process)

6.2. Threads and Fork

How does `fork()` behave in a multi-threaded program?

`fork()` only copies the SINGLE thread that called `fork()` — it does not copy any of the other threads running in the parent process

```
#include <pthread.h>
#include <stdio.h>
#include <unistd.h>
void *worker(void *arg) {
    while (1) {
        printf("Worker thread is running...\n");
```

```
        sleep(1);
    }
    return NULL;
}
int main() {
    pthread_t t;
    pthread_create(&t, NULL, worker, NULL);
    sleep(1);
    pid_t pid = fork();
    if (pid == 0) {
        printf("Child: don't have worker thread\n");
    } else {
        printf("Parent: have worker thread and main\n");
    }
    sleep(2);
    return 0;
}
```

Output:

```
Worker is running...
Worker is running...
Parent: have worker thread and main
Child: don't have worker thread
Worker is running...
Worker is running...
```

→ In the child process, the `worker` thread doesn't exist at all — but if `worker` happened to be holding a mutex exactly when `fork()` occurred, then that mutex remains "locked" in the child process, even though the thread holding the lock (worker) is gone → no one can ever unlock it → permanent deadlock in the child process.

An example: a mutex getting "stuck" after fork

```
The moment fork() occurs:
  Parent process: Main thread (calling fork) + Worker thread (holding mutex X)

  ↓ fork()

  Child process: only the Main thread (which was copied) exists
                 The Worker thread is NOT copied
```

```
                BUT mutex X is still in the "LOCKED" state (its state was copied as-is)
                -> No thread in the child process can unlock X
                -> Anyone in the child process who tries to lock X will wait FOREVER
```

`pthread_atfork()`

This function lets you register callbacks that run automatically around the moment of `fork()`, to safely clean up mutex state:

```
#include <pthread.h>
#include <stdio.h>
pthread_mutex_t lock = PTHREAD_MUTEX_INITIALIZER;
void before_fork(void) {
    pthread_mutex_lock(&lock);
}
void after_fork_in_parent(void) {
    pthread_mutex_unlock(&lock);
}
void after_fork_in_child(void) {
    pthread_mutex_unlock(&lock);
}
int main() {
    pthread_atfork(before_fork, after_fork_in_parent, after_fork_in_child);
    return 0;
}
```

How `pthread_atfork()` works:

- Before fork: Clean up, ensure no one is currently holding the lock.
- After fork in the Parent: Return everything to normal.
- After fork in the Child: Reset the lock to a brand-new state so the Child can use it safely.

If must combine fork() and multi-threading: the child process should call `exec()` immediately after `fork()` (the common `fork() + exec()` pattern) — because `exec()` completely replaces the program's memory, so any "stuck" mutex issues from before no longer matter

6.3. Threads and I/O

File descriptors are shared among threads

Unlike virtual memory (which is independent per process), threads within the same process share a common file descriptor table — meaning that if one thread opens a file, all other threads can see and use that same file.

```
#include <pthread.h>
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>
int fd; /* shared file descriptor*/
void *ghi_du_lieu(void *arg) {
    char *msg = (char *)arg;
    write(fd, msg, strlen(msg));
    return NULL;
}
```

Race condition on the file pointer

The read/write position pointer (file offset) is a SHARED property of the file descriptor, not something private to each thread.

```
/* 2 threads both call*/
write(fd, "AAAA", 4);  /* Thread 1*/
write(fd, "BBBB", 4);  /* Thread 2*/
```

Without synchronization, the two `write()` calls might interleave or overwrite the same offset position, resulting in jumbled data in the file (for example: `"AABBBABA"` instead of the clean `"AAAABBBB"`) — another form of race condition, but this time occurring on an operating system resource (a file) rather than an in-memory variable.

Solution 1: Use a Mutex to protect the I/O operation

```
pthread_mutex_t io_lock = PTHREAD_MUTEX_INITIALIZER;
void *ghi_an_toan(void *arg) {
    char *msg = (char *)arg;
    pthread_mutex_lock(&io_lock);
    write(fd, msg, strlen(msg));
    pthread_mutex_unlock(&io_lock);
    return NULL;
}
```

Only 1 thread may write at a time

Solution 2: Use `pwrite()` / `pread()`

```
#include <unistd.h>
```

```
ssize_t pwrite(int fd, const void *buf, size_t count, off_t offset);
ssize_t pread(int fd, void *buf, size_t count, off_t offset);
```

`pwrite()`/`pread()` take the offset as an explicit parameter, without touching the shared file pointer → each thread can write to its own designated region without needing a mutex lock (as long as the write regions don't overlap)

```
void *ghi_vao_vi_tri(void *arg) {
    int id = *(int *)arg;
    char msg[] = "DATA";
    pwrite(fd, msg, strlen(msg), id * 4);
    return NULL;
}
```

Each thread writes to its own position, no conflict

6.4. The Blocking I/O Issue in Threads

If a thread calls a blocking I/O operation (for example, `read()` from a socket/keyboard that's waiting for data), that thread will "freeze" — but only that specific thread gets blocked; other threads in the same process continue running normally (this is a major difference compared to a single-threaded program, where blocking I/O would freeze the entire program).

→ This is actually one of the main reasons to use threads: create one dedicated thread to handle slow I/O (reading files, reading network sockets), while other threads continue handling other work.

7. Condition Variables and Barriers

7.1. Condition Variables

A mutex alone can't solve "waiting for a condition"

A mutex only solves one thing: "only 1 thread enters the critical section at a time." But there's another common need: 1 thread needs to WAIT until some condition becomes true, and only then continue running.

Classic example problem: Producer-Consumer

- The Consumer thread needs to take data out of a queue to process it
- But if the queue is currently empty, the consumer must wait until the Producer thread puts data into it

Using a busy-checking loop (busy-waiting / polling)

```
pthread_mutex_lock(&lock);
while (queue_rong) {
    pthread_mutex_unlock(&lock);
    /* DO NOTHING, just keep checking again and again*/
    pthread_mutex_lock(&lock);
}
pthread_mutex_unlock(&lock);
```

→ This approach **wastes CPU extremely badly**: the thread keeps locking/unlocking/checking thousands of times per second even though nothing has changed.

The Solution: Condition Variable — WAIT without burning CPU

A Condition Variable lets one thread "sleep" until another thread "wakes it up", signaling that the condition has changed — no CPU is consumed while waiting.

Main functions

```
pthread_cond_t cond = PTHREAD_COND_INITIALIZER;
pthread_mutex_t lock = PTHREAD_MUTEX_INITIALIZER;

pthread_cond_wait(&cond, &lock); /* WAIT: automatically unlocks lock and "sleeps"; re-locks
automatically once woken up*/
pthread_cond_signal(&cond);  /* WAKE UP 1 waiting thread*/
pthread_cond_broadcast(&cond); /* WAKE UP ALL waiting threads*/
```

`pthread_cond_wait()` must always be called while holding the mutex, because internally it automatically:

(1) unlocks the mutex

(2) puts the thread into a sleeping/waiting state

(3) once woken up, re-locks the mutex before returning control to the code to continue running.

Producer-Consumer with a Condition Variable

Lab 4.

```
#include <pthread.h>
#include <stdio.h>
#include <unistd.h>
```

```
int queue_data = -1;
pthread_mutex_t lock = PTHREAD_MUTEX_INITIALIZER;
pthread_cond_t cond = PTHREAD_COND_INITIALIZER;

void *producer(void *arg) {
    sleep(2);
    pthread_mutex_lock(&lock);
    queue_data = 42;
    printf("Producer: da san xuat du lieu = %d\n", queue_data);
    pthread_cond_signal(&cond);
    pthread_mutex_unlock(&lock);
    return NULL;
}
void *consumer(void *arg) {
    pthread_mutex_lock(&lock);
    while (queue_data == -1) {
        printf("Consumer: queue rong, dang cho...\n");
        pthread_cond_wait(&cond, &lock);
    }

    printf("Consumer: da nhan duoc du lieu = %d\n", queue_data);
    pthread_mutex_unlock(&lock);

    return NULL;
}

int main() {
    pthread_t p, c;
    pthread_create(&c, NULL, consumer, NULL);
    pthread_create(&p, NULL, producer, NULL);

    pthread_join(p, NULL);
    pthread_join(c, NULL);
    return 0;
}
```

Output

```
Consumer: queue rong, dang cho...
Producer: da san xuat du lieu = 42
```

```
Consumer: da nhan duoc du lieu = 42
```

→ The consumer doesn't burn any CPU at all while waiting during those 2 seconds — it's genuinely "asleep," and only gets woken up exactly when the producer calls `pthread_cond_signal()`.

Why use `while` instead of `if` to check the condition?

```
while (queue_data == -1) {
    pthread_cond_wait(&cond, &lock);
}
```

instead of

```
if (queue_data == -1) {
    pthread_cond_wait(&cond, &lock);
}
```

Reason: A phenomenon called "spurious wakeup" — a thread in `cond_wait()` can be woken up even though no one called `signal`/`broadcast` (due to how the operating system implements this internally). In addition, if there are multiple consumers waiting at the same time, when `broadcast()` is called, all of them wake up, but perhaps only 1 will actually get the data — the rest need to re-check the condition rather than assuming it's now true by default.

→ Using `while` ensures that: after waking up, the thread always re-checks the actual condition; if it's still not satisfied, it goes back to `wait()` again, avoiding processing incorrect data.

7.2. Barriers

A Barrier is a synchronization point at which all participating threads must stop and wait for each other, until the required number of threads have all reached that point — only then are all of them allowed to continue running at the same time.

→ Unlike a mutex (controls who may enter one spot at a time) or a condition variable (waits for a data condition), a barrier is simply a meeting point among multiple threads.

`pthread_barrier_t`

```
#include <pthread.h>
#include <stdio.h>
#include <unistd.h>
pthread_barrier_t barrier;
void *worker(void *arg) {
    int id = *(int *)arg;
    printf("Thread %d: dang lam giai doan 1...\n", id);
```

```
    sleep(id);
    printf("Thread %d: da xong giai doan 1, cho o barrier...\n", id);
    pthread_barrier_wait(&barrier); /* WAIT until ENOUGH threads have arrived here*/
    printf("Thread %d: tat ca da san sang, bat dau giai doan 2!\n", id);
    return NULL;
}
int main() {
    pthread_t threads[3];
    int ids[3] = {1, 2, 3};
    pthread_barrier_init(&barrier, NULL, 3);
    for (int i = 0; i < 3; i++)
        pthread_create(&threads[i], NULL, worker, &ids[i]);
    for (int i = 0; i < 3; i++)
        pthread_join(threads[i], NULL);
    pthread_barrier_destroy(&barrier);
    return 0;
}
```

Output

```
Thread 2: dang lam giai doan 1...
Thread 3: dang lam giai doan 1...
Thread 1: dang lam giai doan 1...
Thread 1: da xong giai doan 1, cho o barrier...
Thread 2: da xong giai doan 1, cho o barrier...
Thread 3: da xong giai doan 1, cho o barrier...
Thread 3: tat ca da san sang, bat dau giai doan 2!
Thread 1: tat ca da san sang, bat dau giai doan 2!
Thread 2: tat ca da san sang, bat dau giai doan 2!
```

→ Even though `Thread 1` finishes its work earliest (after 1 second), it still has to wait until `Thread 3` (the slowest, after 3 seconds) also reaches the barrier — only then do all 3 print the "starting phase 2" line at nearly the same time.

Main function

Function	Meaning
`pthread_barrier_init(&barrier, NULL, N)`	Initializes the barrier, specifying that exactly N threads must arrive before they can pass through

Function	Meaning
`pthread_barrier_wait(&barrier)`	The thread calling this function will wait here until N threads have all called this function
`pthread_barrier_destroy(&barrier)`	Destroys the barrier once it's no longer needed

8. Semaphores for Synchronization vs. Mutexes for Mutual Exclusion

8.1. What is a Semaphore?

A Semaphore is a special counting variable used to control how many threads are allowed to access a resource concurrently — not limited to "1," but can be N (any number specify).

A semaphore has 2 main operations:

- `wait()` (also called `P`, `sem_wait`): decreases the counter by 1. If the counter is currently 0, the thread calling this function will be blocked, waiting until someone "returns a slot"
- `signal()` (also called `V`, `sem_post`): increases the counter by 1, waking up 1 waiting thread (if any)

8.2. Using Semaphores in POSIX (`sem_t`)

```
#include <semaphore.h>


sem_t sem;
sem_init(&sem, 0, N);   /* N = initial value */
                         /* the middle parameter (0) = shared among threads of the SAME
process*/
sem_wait(&sem);          /* request 1 slot - wait if no slots remain*/
// ... use the resource ...
sem_post(&sem);          /* return 1 slot*/
sem_destroy(&sem);       /* destroy when no longer needed*/
```

8.3. Example

```
#include <pthread.h>
#include <semaphore.h>
#include <stdio.h>
#include <unistd.h>
```

```
sem_t parking_lot;

void *car_enters(void *arg) {
    int id = *(int *)arg;
    printf("Car %d is waiting for a spot...\n", id);
    sem_wait(&parking_lot);
    printf("Car %d: ENTERED THE LOT, parking...\n", id);
    sleep(2); // simulate parking time
    printf("Car %d: leaving the lot, returning 1 spot\n", id);
    sem_post(&parking_lot);
    return NULL;
}

int main() {
    sem_init(&parking_lot, 0, 2);

    pthread_t cars[5];
    int ids[5] = {1, 2, 3, 4, 5};

    for (int i = 0; i < 5; i++)
        pthread_create(&cars[i], NULL, car_enters, &ids[i]);

    for (int i = 0; i < 5; i++)
        pthread_join(cars[i], NULL);

    sem_destroy(&parking_lot);
    return 0;
}
```

→ With `sem_init(&parking_lot, 0, 2)`, only 2 cars can be in the lot at the same time, even though 5 cars arrive together. The remaining 3 cars must wait until another car leaves (`sem_post()`) before they can enter.

8.4. The core difference between Mutex and Semaphore

Criteria	Mutex	Semaphore
Main purpose	Mutual Exclusion — protecting a critical section	Synchronization — coordinating/signaling between threads, controlling the number of accesses

Criteria	Mutex	Semaphore
Counter value	Only 2 states: locked / unlocked (like 0 and 1)	Can be any integer (N resources)
Who may unlock	REQUIRED: whichever thread called `lock()` must be the one to call `unlock()` (there's a concept of "ownership")	NOT required: thread A can `wait()`, while thread B is the one who `signal()`s — there is no "owner"
What it's for	Ensuring data isn't modified by 2 threads at the same time	Ensuring at most N threads use a resource, or "signaling" that an event has occurred

```
/* MUTEX - has an "owner"*/
pthread_mutex_lock(&lock);   /* Thread A locks it*/

pthread_mutex_unlock(&lock); /* MUST be Thread A that unlocks it - Thread B calling unlock()
is WRONG/undefined behavior*/
```

```
/* SEMAPHORE - has NO "owner"*/
sem_wait(&sem);   /* Thread A "requests a slot"*/

/* Thread B (COMPLETELY DIFFERENT, unrelated to Thread A's earlier wait)*/
sem_post(&sem);   /* Thread B "returns a slot" - COMPLETELY VALID, doesn't need to be Thread A
itself*/
```

→ Precisely because of this property, a semaphore can be used to "signal" between 2 different threads — something a mutex cannot do properly.

8.5. Using a Semaphore for SIGNALING — something a mutex can't do

This is the most important application distinguishing semaphores from mutexes: using a semaphore so that one thread can notify another thread that "I've finished my work

```
#include <pthread.h>
#include <semaphore.h>
#include <stdio.h>
#include <unistd.h>

sem_t sem_bao_hieu;

void *thread_A(void *arg) {
```

```
    printf("A: dang chuan bi du lieu...\n");
    sleep(2);
    printf("A: xong roi, bao cho B biet!\n");
    sem_post(&sem_bao_hieu);

    return NULL;
}

void *thread_B(void *arg) {
    printf("B: dang cho A bao hieu...\n");
    sem_wait(&sem_bao_hieu);
    printf("B: da nhan tin hieu, tiep tuc lam viec!\n");

    return NULL;
}

int main() {
    sem_init(&sem_bao_hieu, 0, 0);

    pthread_t a, b;
    pthread_create(&b, NULL, thread_B, NULL);
    pthread_create(&a, NULL, thread_A, NULL);

    pthread_join(a, NULL);
    pthread_join(b, NULL);

    sem_destroy(&sem_bao_hieu);
    return 0;
}
```

→ `sem_init(&sem_bao_hieu, 0, 0)` initializes the semaphore with a value of 0 — meaning B will always have to wait until A calls `sem_post()`. This is the "signaling" (one-way notification) pattern, quite different from a mutex (which is meant for protecting data, not for signaling events between different threads).

8.6. When to use which?

- **Use a Mutex** when:
  - Need to protect a shared piece of data/resource, ensuring only 1 thread modifies it at a time — this is the classic mutual exclusion problem (like the `counter++` example from the first race-condition document)

- **Use a Semaphore** when:
  - Need to limit N threads accessing something concurrently (e.g., a parking lot with N spots, limiting N connections to a database)
  - Need to signal an even* between different threads (one thread notifying another that "the work is done"), without an actual need to "lock data"

9. Deadlock: Necessary Conditions, Prevention, Avoidance, Detection

9.1. What Is a Deadlock?

A deadlock is a situation where two or more threads (or processes) are each waiting for a resource held by another, and none of them can proceed — they wait for each other forever.

```
Thread A: holds Lock 1, waiting for Lock 2
Thread B: holds Lock 2, waiting for Lock 1
→ Neither can ever continue.
```

This has already come up in earlier documents: a signal handler waiting on a mutex the interrupted thread was holding, and a mutex left permanently locked in a child process after `fork()`. Both are specific instances of the general deadlock problem covered here.

9.2. The Four Necessary Conditions

A deadlock can only occur if all four of the following conditions hold at the same time. Removing any single one of them makes deadlock impossible.

Mutual Exclusion

At least one resource must be held in a non-shareable way — only one thread can use it at a time (e.g. a mutex-protected resource).

Hold and wait

A thread is holding at least one resource while simultaneously waiting to acquire additional resources that are currently held by other threads.

```
pthread_mutex_lock(&lock1);   /* holding lock1*/
pthread_mutex_lock(&lock2);   /* ...while waiting for lock2*/
```

No Preemption

A resource can only be released voluntarily by the thread holding it — no other thread or the OS can forcibly take it away.

Circular Wait

There exists a circular chain of threads, where each thread is waiting for a resource held by the next thread in the chain.

```
Thread A waits for a resource held by Thread B
Thread B waits for a resource held by Thread C
Thread C waits for a resource held by Thread A
→ a cycle: A → B → C → A
```

Example: 2 Locks, 2 Threads

Lab 5. Reproduce a deadlock between two threads acquiring two locks in opposite order.

```
#include <pthread.h>
#include <stdio.h>
#include <unistd.h>

pthread_mutex_t lock1 = PTHREAD_MUTEX_INITIALIZER;
pthread_mutex_t lock2 = PTHREAD_MUTEX_INITIALIZER;

void *thread_A(void *arg) {
    pthread_mutex_lock(&lock1);
    printf("A: locked lock1, trying to lock lock2...\n");
    sleep(1);
    pthread_mutex_lock(&lock2); /* A now waits for lock2, held by B*/
    printf("A: got both locks\n");

    pthread_mutex_unlock(&lock2);
    pthread_mutex_unlock(&lock1);
    return NULL;
}

void *thread_B(void *arg) {
    pthread_mutex_lock(&lock2);
    printf("B: locked lock2, trying to lock lock1...\n");
    sleep(1);
    pthread_mutex_lock(&lock1); /* B now waits for lock1, held by A*/
    printf("B: got both locks\n");
```

```
    pthread_mutex_unlock(&lock1);
    pthread_mutex_unlock(&lock2);
    return NULL;
}

int main() {
    pthread_t a, b;
    pthread_create(&a, NULL, thread_A, NULL);
    pthread_create(&b, NULL, thread_B, NULL);
    pthread_join(a, NULL);
    pthread_join(b, NULL);
    return 0;
}
```

Output:

```
A: locked lock1, trying to lock lock2...
B: locked lock2, trying to lock lock1...
```

The program hangs forever after these two lines — it never terminates

→ All four conditions are present: mutual exclusion (each mutex is exclusive), hold-and-wait (each thread holds one lock while waiting for the other), no preemption (neither lock can be forcibly taken), and circular wait (A waits on B, B waits on A).

9.3. Deadlock Prevention

Prevention means structurally guaranteeing that at least one of the four conditions can never occur, so deadlock is impossible by design.

Attack Mutual Exclusion

Make resources shareable where possible (e.g. read-only data, or lock-free/atomic operations instead of a mutex). Not always feasible — some resources are inherently exclusive (e.g. writing to the same file region).

Attack Hold and Wait — acquire all locks at once

Require a thread to request all the resources it will need before starting work, and only proceed once it has all of them.

Attack No Preemption — allow forced release

If a thread already holding some resources requests another one and can't get it immediately, it releases everything it's currently holding, then tries again later.

Attack Circular Wait — Lock Ordering

Assign a fixed global order to every lock, and require every thread to acquire locks only in that order, never in the reverse order.

Lab 5. Consistent Lock Ordering

```
#include <pthread.h>
#include <stdio.h>
#include <unistd.h>

pthread_mutex_t lock1 = PTHREAD_MUTEX_INITIALIZER;
pthread_mutex_t lock2 = PTHREAD_MUTEX_INITIALIZER;
/* RULE: lock1 must always be acquired before lock2, in every thread, no exceptions*/

void *thread_A(void *arg) {
    pthread_mutex_lock(&lock1);
    sleep(1);
    pthread_mutex_lock(&lock2);
    printf("A: got both locks\n");

    pthread_mutex_unlock(&lock2);
    pthread_mutex_unlock(&lock1);
    return NULL;
}
void *thread_B(void *arg) {
    pthread_mutex_lock(&lock1);
    sleep(1);
    pthread_mutex_lock(&lock2);
    printf("B: got both locks\n");
    pthread_mutex_unlock(&lock2);
    pthread_mutex_unlock(&lock1);
    return NULL;
}

int main() {
    pthread_t a, b;
    pthread_create(&a, NULL, thread_A, NULL);
```

```
    pthread_create(&b, NULL, thread_B, NULL);
    pthread_join(a, NULL);
    pthread_join(b, NULL);
    return 0;
}
```

→ Now, whichever thread gets `lock1` first, the other thread will simply block waiting for `lock1` (not `lock2`), so there's no circular chain — no deadlock is possible, since a cycle can never form when every thread requests resources in the same fixed order.

In real systems with many locks (not just two), a common convention is to order locks by a stable property, such as a memory address or an assigned numeric ID, and always acquire them from lowest to highest.

9.4. Deadlock Avoidance

Avoidance is different from prevention: instead of structurally forbidding one of the four conditions, the system makes runtime decisions about whether granting a resource request is safe, based on information about future needs.

The Banker's Algorithm

The classic avoidance algorithm. Each process must declare in advance the maximum amount of each resource it might ever need. Before granting a request, the system simulates granting it and checks whether the resulting state is still a safe state — meaning there exists at least one order in which all processes could still finish given the resources currently available.

- If granting the request keeps the system in a safe state → grant it
- If granting the request could lead to an unsafe state (a state where a deadlock might become unavoidable) → the request is delayed, even if the resource is currently free

Trade-offs of Avoidance

- Requires knowing resource needs in advance, which is often unrealistic in real programs
- Higher runtime overhead than prevention, since every request requires a safety check
- Rarely implemented directly in general-purpose OS kernels or everyday application code; lock ordering (prevention) is used far more often in practice
- More relevant in specialized systems (e.g. embedded systems with a fixed, known set of resource requirements)

9.5. Deadlock Detection and Recovery

Instead of preventing or avoiding deadlock, this strategy allows it to potentially happen, but periodically checks whether it has occurred, and recovers if so.

Detection via Resource-Allocation / Wait-For Graph

The system builds a graph where each thread/process is a node, and a directed edge from A to B means "A is waiting for a resource held by B." A **cycle** in this graph indicates a deadlock.

```
A → B → C → A     (cycle detected → deadlock exists)
```

Detection can be run periodically (e.g. every few seconds) or triggered when a resource request has been waiting unusually long.

Detecting a Stuck Program from the Outside (practical tools)

- `gdb`: attach to a running process (`gdb -p <pid>`) and run `thread apply all bt` to get a backtrace of every thread — threads stuck on `pthread_mutex_lock()` in a cycle are visible this way
- `strace`: shows if a process is blocked in a system call (e.g. `futex`, which pthread mutexes are built on)
- Watchdog timers: a thread that hasn't reported progress within an expected window is flagged as potentially deadlocked

Recovery Strategies

Once a deadlock is detected, recovery generally means breaking the cycle by force:

- Kill one or more threads/processes involved in the cycle, forcing them to release their resources (simplest, but loses whatever work those threads were doing)
- Preempt a resource: forcibly take a resource away from one of the threads in the cycle and give it to another, then roll the first thread back to a safe point
- Rollback: if the system periodically checkpoints process state, roll one of the deadlocked processes back to an earlier checkpoint before it acquired the contested resource

Strategy	Idea	Cost	Where it's typically used
Prevention	Structurally eliminate one of the 4 necessary conditions (most commonly: enforce a fixed lock order)	Low runtime cost, but requires careful design discipline	Everyday multi-threaded application code
Avoidance	Make runtime decisions using advance knowledge of future resource needs (e.g. Banker's Algorithm)	Higher runtime overhead, needs advance declarations	Specialized/embedded systems with known resource bounds
Detection & Recovery	Let deadlock potentially happen, detect it (e.g. cycle in a wait-for graph), then break it	Deadlock can actually occur temporarily; recovery may lose work	OS-level resource managers, databases, systems with timeouts/watchdogs

---

Revision #4
Created 2026-08-27 02:57:56 UTC by Huyen Nguyen
Updated 2026-09-04 01:40:59 UTC by Huyen Nguyen